Scaling GPU KV-Cache Efficiency and Inference Concurrency for Autonomous Agents with HashedContext

Rahat Hasan
Akkhar-Labs
Bogura, Bangladesh
rahathasan5773@gmail.com

Abstract

Autonomous agentic coding systems routinely transmit source files to Large Language Models (LLMs) across multi-turn software development workflows. In conventional architectures, unchanged files are re-read from storage and redundantly re-serialized into append-only conversation histories on every interaction step. This produces an asymptotic token growth of $\mathcal{O}(M \times N \times P)$, which severely exacerbates the GPU High-Bandwidth Memory (HBM) "memory wall" by saturating key-value (KV) cache allocations and degrading multi-tenant serving concurrency.

We introduce **HashedContext**, a content-addressed context architecture that eliminates redundant file transmission and optimizes physical KV-cache utilization across multi-turn agent sessions. HashedContext couples four complementary mechanisms: (1) SHA-256 hash-gated file state tracking, (2) historical conversation deduplication via compact references (`[HASH_STUB]`), (3) attention-anchored authoritative tail rebasing, and (4) on-demand historical state resolution from a local content-addressed vault, and supports a bimodal deployment topology operable either edge-natively within client editors or transparently at server-side inference gateways.

Empirical evaluations on production inference endpoints (DeepSeek V4) demonstrate an immediate **57.7% to 67.2% reduction in active prompt tokens** on basic 4-to-5 turn workflows, with asymptotic savings mathematically bounded to **90%–97%** on extended multi-file refactoring sessions. We observe no detectable degradation in model reasoning accuracy or diff resolution fidelity. By restricting full file materialization to a single authoritative prompt tail, HashedContext transforms context accumulation from an unbounded quadratic trajectory into a bounded linear footprint. This reduction directly relieves GPU high-bandwidth memory (HBM) saturation and expands concurrent serving capacity for agent workloads.

1 Introduction

The emergence of autonomous agentic coding frameworks (e.g., Devin, Claude Code, Cursor, Cline) has shifted large language model interaction from single-turn completion to long-horizon, multi-turn state mutation. In a representative debugging session, an autonomous agent iteratively inspects source trees, formulates hypotheses, executes edits, evaluates test suites, and verifies syntax over dozens of sequential conversational turns.

Despite significant operational capabilities, modern agentic systems suffer from a critical architectural inefficiency: **unbounded redundant context serialization**. Because autonomous agents lack cryptographic confidence in file system immutability across turns, existing tools routinely re-read and append full file contents into conversation histories prior to every action. For a repository working set of M files averaging N tokens over P turns, the cumulative prompt load scales as $\mathcal{O}(M \times N \times P)$.

This excessive re-transmission causes profound systemic strain:

1. **KV-Cache Saturation and VRAM Waste:** Modern model serving engines allocate gigabytes of scarce high-bandwidth memory (HBM3) to retain Key-Value (KV) cache vectors. At scale, long contexts saturate GPU memory capacity, forcing frequent context eviction or high-latency offloading to host RAM and NVMe storage.

2. **Attention Dilution and Version Regression:** Transformer self-attention mechanisms exhibit performance degradation (the "lost-in-the-middle" phenomenon) when multiple conflicting, partially edited copies of the same source file populate the prompt history. Models frequently attend to historical bugs already resolved in earlier turns.

3. **Memory-Bandwidth Inference Throttling:** Autoregressive decoding speed is fundamentally memory-bandwidth bound. Streaming hundreds of thousands of historical Key/Value vectors from HBM into SRAM on every generated token substantially increases Time-Per-Output-Token (TPOT).

While server-side techniques like PagedAttention [1] have revolutionized *spatial memory allocation* by partitioning physical VRAM into non-contiguous blocks, they

do not reduce the *temporal redundancy* of the input sequence itself. A 100,000-token prompt still consumes 100,000 tokens of KV-cache, regardless of how efficiently those blocks are mapped.

To address this challenge, we present **HashedContext**, a content-addressed prompt architecture designed to decouple session horizon length from token memory footprint. Inspired by Content-Addressed Storage (CAS) principles employed in cryptographic state tracking systems, HashedContext replaces redundant historical file observations with lightweight cryptographic hash stubs (`[HASH_STUB]`), while rebasing the single authoritative, live representation of the files at the prompt's tail.

Contributions. Our primary contributions are:

- **The HashedContext Architecture:** A formal protocol combining SHA-256 content-addressing, historical stubbing, and attention-anchored tail rebasing for multi-turn agent workflows.
- **Asymptotic Complexity Reduction:** A mathematical derivation demonstrating the collapse of file transmission overhead from $\mathcal{O}(M \times N \times P)$ to $\mathcal{O}((M \times N) + (D \times P))$, where D represents modified token volume per turn.
- **Bimodal Deployment Topology:** A dual-mode architectural blueprint that enables execution either on the client side (IDE extensions, agent CLI) or unilaterally as a transparent server-side inference gateway filter.
- **Empirical Telemetry Verification:** Production validation over frontier endpoints demonstrating a 67.19% prompt token reduction on standard 5-turn micro-sessions with zero loss of model comprehension or audit accuracy.

2 Background and Motivation

2.1 The Memory Wall in LLM Serving

During transformer autoregressive generation, keys and values computed for prior tokens are stored in the KV-cache to avoid recomputation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1}$$

For a model with L layers, H_{kv} attention heads, and hidden dimension d_h, the physical memory consumed by a sequence of length S at precision b bytes is:

$$\text{Mem}_{\text{KV}} = 2 \times L \times H_{kv} \times d_h \times b \times S \tag{2}$$

For a modern 70B parameter dense model or mixture-of-experts (MoE) architecture (e.g., DeepSeek, Llama-3-70B) utilizing 80 layers and Grouped-Query Attention (GQA) with FP8 precision ($b = 1$), every token requires approximately 160 KB of active KV-cache. A conventional 100,000-token prompt locks ≈ 16 GB of high-speed VRAM per user. In multi-tenant environments, a cluster of 80GB GPUs quickly saturates its memory capacity, reducing batch concurrency to single digits.

2.2 The Failure of Conventional Mitigations

Existing optimizations fail to fully resolve multi-turn agentic bloat:

- **Prefix Caching:** While prompt caching saves compute FLOPs by matching static prefixes, it *does not reduce VRAM consumption.* The cached states must remain pinned in HBM, consuming identical memory. Furthermore, when agents modify files mid-session, prefix matching is often partially invalidated.
- **Context Window Truncation (Sliding Window):** Dropping early messages destroys conversational continuity and breaks repository awareness.
- **Iterative Summarization:** Summarizing code history via LLM incurs additional API costs, introduces latency, and frequently drops critical structural details (variable names, types, line numbers).

2.3 Input-Output Asymmetry and Amdahl's Leverage in Agentic Workloads

Autonomous agentic coding workflows exhibit an extreme asymmetry between input ingestion and output generation. In a typical multi-file refactoring session spanning P turns, each turn loads M files averaging N tokens alongside dialogue history, while producing only a targeted diff or edit of $D_k \ll M \cdot N$ output tokens. Empirically, the ratio of ingested context tokens to generated output tokens frequently exceeds 300 : 1 to 400 : 1 in repository-level coding sessions.

This asymmetry reframes the optimization priority under Amdahl's Law [5]. Let f represent the fraction of end-to-end latency attributable to input prefill and KV-cache management versus output decoding. In conventional agentic workloads:

$$f = \frac{T_{\text{prefill}} + T_{\text{KV}}}{T_{\text{prefill}} + T_{\text{KV}} + T_{\text{decode}}} \tag{3}$$

where T_{prefill} scales with total input sequence length S, T_{KV} scales with memory saturation and cache thrashing, and T_{decode} scales linearly with generated token count G. Under realistic token ratios ($S/G \approx 300$), the input-dominated fraction satisfies $f > 0.99$. Amdahl's Law then dictates that the maximum achievable speedup $\mathcal{S}$ from optimizing exclusively the decode path is bounded by:

$$\mathcal{S}_{\text{decode-only}} \leq \frac{1}{1 - f} \tag{4}$$

which, for $f > 0.99$, yields an upper bound of approximately 100×—yet decode optimizations (speculative decoding, KV-quantization) attack at most 1% of the latency budget. By contrast, reducing input redundancy—the core mechanism of HashedContext—directly addresses the dominant f term, yielding substantially higher system-level returns. Prior literature's focus on accelerating autoregressive decoding leaves the primary latency, token cost, and memory saturation bottleneck in agentic systems—repetitive prompt prefill and redundant KV-cache residency—largely unaddressed.

2.4 In-Situ Proactive Deduplication versus Post-Hoc Reactive Compaction

The preceding analysis establishes input redundancy as the dominant performance bottleneck in agentic coding systems. We now examine the structural divergence between reactive post-hoc compaction strategies (recursive LLM summarization, sliding-window truncation, rolling buffers) and HashedContext's proactive in-situ deduplication across three formal dimensions.

Secondary Computational Overhead

Existing approaches to context compression invoke auxiliary LLM calls to summarize or condense historical context after redundancy has already been serialized. Let T_{compact} denote total session wall time under a post-hoc strategy:

$$T_{\text{compact}} = T_{\text{session}} + \sum_{i=1}^{P} \Delta T_{\text{aux}}^{(i)} \tag{5}$$

where each $\Delta T_{\text{aux}}^{(i)}$ represents the latency of an auxiliary prefill-decode cycle for compaction at turn i. These cycles incur both additional FLOPs (forward passes over the accumulated context) and API-level token costs, often amplifying total inference expenditure by 2–5× over the base session [8]. Furthermore, recursive summarization requires the model to attend to the *entire* historical context during each compaction pass, exacerbating the very memory saturation it purports to alleviate.

By contrast, HashedContext operates at the transmission boundary on the CPU, computing a SHA-256 hash of each file observation prior to serialization. No auxiliary model invocation is required: $\Delta T_{\text{aux}} = 0$ for all turns. The marginal cost per file reduces to a single hash computation ($\approx$ 2–5 μs per MB on modern x86-64), vanishing relative to LLM prefill latency.

Syntactic Entropy Loss in Code Contexts

Natural language discourse tolerates semantic summarization: a human reader can follow a compressed narrative that omits exact word choices while preserving intended meaning. Formal software source trees, however, require exact byte-level AST reproducibility. Let $\mathcal{T}_{\text{code}}$ denote the ground-truth abstract syntax tree of a source file, and $\mathcal{S}_{\text{summary}}$ denote the summary produced by a reactive compaction pass. The conditional syntactic entropy introduced by summarization is:

$$\mathcal{H}(\mathcal{T}_{\text{code}} \mid \mathcal{S}_{\text{summary}}) = -\sum_{t \in \mathcal{T}_{\text{code}}} P(t \mid \mathcal{S}_{\text{summary}}) \log P(t \mid \mathcal{S}_{\text{summary}}) > 0 \tag{6}$$

This non-zero entropy captures the probability that structural elements—variable declarations, type annotations, import paths, line-level diffs—are irrecoverably lost or hallucinated during condensation. In practice, recursive summarization of code histories has been shown to induce downstream compilation regressions and test failures, as the model operates on an imprecise internal reconstruction rather than the authoritative source [9].

HashedContext preserves lossless byte identity: because file contents are transmitted once and referenced via cryptographic stub thereafter, the LLM's working state always reflects the exact authoritative source. Under this regime, $\mathcal{H}(\mathcal{T}_{\text{code}} \mid \mathcal{S}_{\text{stub}}) = 0$, as the hash stub deterministically resolves to the unmodified original without information loss.

Cumulative VRAM Dwell Time

A critical yet underexamined property of long-running agent sessions is the cumulative memory allocation over the session lifecycle. The peak memory integral captures the total VRAM occupancy integrated across time:

$$\mathcal{M}_{\text{dwell}} = \int_0^{T_{\text{session}}} \text{Mem}_{\text{KV}}(t)\, dt \tag{7}$$

Under post-hoc compaction, $\text{Mem}_{\text{KV}}(t)$ follows a sawtooth profile: each turn appends $M \times N$ new tokens (and their KV vectors) to the cache, saturating toward the threshold $S_{\text{threshold}}$ before a compaction event releases a fraction of the allocated memory. For the majority of interaction turns, however, $\text{Mem}_{\text{KV}}(t)$ dwells at near-saturation levels:

$$\frac{1}{T_{\text{session}}} \int_0^{T_{\text{session}}} \text{Mem}_{\text{KV}}(t)\, dt \approx S_{\text{threshold}} - \epsilon \tag{8}$$

where ϵ is the small transient reduction achieved by each compaction pass. This sustained pressure constrains GPU batch concurrency and forces frequent cache offloading to host memory, degrading tail latency.

HashedContext bounds $\text{Mem}_{\text{KV}}(t)$ strictly to the cardinality of the live working set $\mathcal{W}(t)$—the set of files actively observed or modified in the current turn—plus dialogue overhead. Because historical file content is replaced by fixed-length hash stubs, the KV-cache footprint per file collapses from $\mathcal{O}(N)$ to $\mathcal{O}(1)$. The dwell integral satisfies:

$$\mathcal{M}_{\text{dwell}} \leq |\mathcal{W}(t)| \cdot C_{\text{stub}} + D_{\text{dialogue}} \tag{9}$$

where C_{stub} is the constant KV-cache cost of a hash stub token (tens of bytes) and D_{dialogue} accounts for the conversational turn history. This bound holds uniformly across the entire session lifecycle, eliminating the cumulative memory pressure that characterizes reactive compaction approaches.

In summary, while recursive summarization and sliding-window truncation provide tractable heuristics for open-ended natural language dialogue, deterministic software engineering workflows—where exact byte-level reproducibility, bounded VRAM occupancy, and zero auxiliary compute overhead are required—necessitate the in-situ content-addressed deduplication that HashedContext provides.

3 The HashedContext Architecture

HashedContext operates on a foundational principle: *The conversation history of an agent should serve as a state transition log, not a verbatim filesystem archive.*

3.1 Cryptographic Hash Stubs

Instead of re-emitting large file bodies into prompt messages, HashedContext abstracts unchanged files into standardized, compact textual stubs termed `HashStubs`:

Listing 1: Standardized HashStub Structure

```
[HASH_STUB]
path: src/auth/session.ts
sha256: e4b2c91f7a62d88d3e91b40c21a7f05b
       8823190dfca91283c4b58e99128741aa
state: CACHED
[/HASH_STUB]
```

Each stub consumes fewer than 40 tokens while uniquely referencing the exact byte content of the file via its SHA-256 digest.

3.2 Attention-Anchored Authoritative Rebasing

Transformers exhibit strong positional recency bias: tokens placed near the end of the context window receive higher attention allocation than tokens situated in the middle of long prompts.

HashedContext exploits this inductive bias by maintaining exactly **one authoritative, fully materialized copy** of each referenced file, anchored strictly at the tail of the prompt assembly immediately preceding the model's generation head:

$$P_{\text{assembled}} = S_{\text{sys}} \circ \mathcal{H}_{\text{stubbed}} \circ U_{\text{curr}} \circ \mathcal{F}_{\text{authoritative}} \tag{10}$$

where S_{sys} is the system instruction, $\mathcal{H}_{\text{stubbed}}$ represents historical turns with files replaced by stubs, U_{curr} is the active user turn, and $\mathcal{F}_{\text{authoritative}}$ is the current canonical file state.

This achieves two objectives:

1. **Single Source of Truth:** The model cannot attend to obsolete syntax from earlier turns, eliminating multi-version hallucination regressions.
2. **Minimal Attention Overhead:** The historical transcript remains lightweight, allowing the attention mechanism to focus its parameters over the actual change boundaries.

Self-Attesting Live Payloads. When the authoritative file state $\mathcal{F}_{\text{authoritative}}$ is appended to the prompt tail, its structure directly encodes its own cryptographic digest within the metadata header:

$$\mathcal{F}_{\text{authoritative}} = \langle \text{path},\ \text{sha256(content)},\ \text{content} \rangle \tag{11}$$

where sha256(content) is the SHA-256 digest of the byte-exact file body computed at assembly time. This self-describing structure enables the model's self-attention heads to perform an $\mathcal{O}(1)$ cryptographic equivalence check entirely within the forward pass.

Systems consequence. When an agent or user queries whether a file has changed across turns, the model compares the live header hash—embedded in $\mathcal{F}_{\text{authoritative}}$ at the tail—directly against historical stub entries `[HASH_STUB]` within its attention matrix. No external retrieval tool, filesystem stat call, or secondary storage lookup is required. The equality check reduces to a token-level string comparison within the already-attended context window, eliminating round-trip latency and preserving the zero-retrieval invariant of the architecture.

3.3 Action Narrative Preservation and Semantic Losslessness

HashedContext enforces an event-sourced invariant that distinguishes two fundamental classes of content: **State Representations** and **Action Narratives**.

The State-Action Dichotomy. A *state representation* is a static file body—an immutable byte sequence that, when unchanged across turns, carries zero new information. HashedContext deduplicates these into hash stubs. An *action narrative* is the agent's reasoning trace: tool invocations, natural-language plans, edit diffs, test output, and validation logs. These constitute the permanent semantic ledger of the session. HashedContext never compresses, truncates, or stubs action narratives; they remain in raw, uncompressed textual form throughout the full interaction horizon.

Eliminating Boilerplate Attention Dilution. By stripping unchanged file bodies from the historical transcript while preserving every reasoning token and semantic

Conventional Architecture:

`[Turn 1: Full File A (70K)]` → `[Turn 2: Edit Plan + Full File A (70K)]` → `[Turn 3: Test + Full File A (70K)]`

Total Context = 210,000 tokens

HashedContext Architecture:

`[Turn 1: [HASH_STUB A] (35T)]` → `[Turn 2: Edit Plan + [HASH_STUB A] (35T)]` → `[Turn 3: Authoritative File A (70K)]`

Total Context = 70,070 tokens (**−66.6% reduction**)

Figure 1: Comparison of Context Assembly Topologies between Conventional Append-Only Logs and HashedContext.

diff, HashedContext focuses the model's finite attention budget on the actual mutations across turns. The consequence is twofold:

1. **Trajectory fidelity is preserved:** The full chronological chain of agent decisions, tool outputs, and validation results remains accessible for audit, rollback analysis, and task resumption.
2. **Semantic noise is suppressed:** Boilerplate file dumps that would otherwise dilute softmax probability mass across thousands of redundant tokens are replaced by compact stubs, forcing the attention mechanism to concentrate on the informative boundaries.

This design guarantees that reasoning accuracy and fine-grained diff tracking remain unimpaired across dozens of interaction turns, a result confirmed by the empirical needle-in-a-Haystack verification reported in Section 11.

3.4 Formal Assembly Algorithm

Algorithm 1 details the prompt reconstruction process executed per interaction turn.

Algorithm 1 HashedContext Prompt Reconstruction

Require: Conversation history $\mathcal{M}$, Active Files $\mathcal{F}$, Cache $\mathcal{C}$, Vault $\mathcal{V}$
Ensure: Compact, Authoritative Prompt P

$\mathcal{H}_{\text{compact}} \leftarrow [\,]$
for each message $m \in \mathcal{M}_{0\ldots|\mathcal{M}|-2}$ **do**
  $m' \leftarrow m$
  for each file block $f \in m.\text{files}$ **do**
    $h \leftarrow \text{SHA256}(f.\text{content})$
    $\mathcal{V}[h] \leftarrow f.\text{content}$ ▷ Vault historical payload
    $\mathcal{C}[f.\text{path}] \leftarrow h$
    $m' \leftarrow \text{ReplaceWithStub}(m', f.\text{path}, h)$
  end for
  $\mathcal{H}_{\text{compact}}.\text{append}(m')$
end for
$P_{\text{tail}} \leftarrow [\,]$
for each file $f \in \mathcal{F}_{\text{active}}$ **do**
  $P_{\text{tail}}.\text{append}(\text{FormatLiveState}(f))$
end for
$P \leftarrow \mathcal{H}_{\text{compact}} \circ \mathcal{M}_{\text{last}} \circ P_{\text{tail}}$
return P
▷ On-demand: $\mathcal{R}(h) \rightarrow \mathcal{V}[h]$ ▷ See §3.5

3.5 Hierarchical Context Tiering and On-Demand State Resolution

HashedContext partitions context storage into two hierarchical tiers to decouple active inference memory from historical archive bulk:

1. **Primary In-Context Working Set:** Active live files and recent instructions are maintained resident in the generation context within GPU KV-cache. This tier corresponds to the prompt tail $\mathcal{F}_{\text{authoritative}}$ in Equation (1) and encompasses the current file system snapshot immediately visible to the attention mechanism.
2. **Secondary Content-Addressed Vault ($\mathcal{V}$):** All historical file payloads are persisted in a local content-addressed store, the *HashVault*, indexed deterministically by their SHA-256 digest. Each vault entry is a mapping $\mathcal{V}[\text{hash}] \mapsto \text{byte_content}$, enabling instant retrieval without re-reading from the filesystem or re-requesting from the agent.

On-Demand Historical Resolution. In nominal operation (>99% of turns), only the stub representation of historical files enters the prompt assembly. However, when an agentic task requires cross-turn analysis—comparing the current file state against a version observed k turns ago, or inspecting a diff across historical variants—the system resolves the referenced `[HASH_STUB]` from the local HashVault. Formally, given a stub referencing hash h, the resolution function $\mathcal{R}$ retrieves the verified byte content:

$$\mathcal{R}(h) = \begin{cases} \mathcal{V}[h] & \text{if } h \in \text{dom}(\mathcal{V}) \\ \emptyset & \text{otherwise (cache miss)} \end{cases} \tag{12}$$

The resolved content is injected into the prompt payload for that single evaluation cycle only; it does not persist

across subsequent turns, thereby avoiding permanent KV-cache pressure. This mechanism guarantees **lossless historical state preservation under negligible collision probability**: full historical state is always recoverable on demand via SHA-256 content addressing ($\mathcal{O}(2^{-256})$ collision bound) with sub-millisecond local disk latency, without permanently penalizing active GPU VRAM.

Zero Information Loss Guarantee. Because every historical file version is content-addressed by its SHA-256 digest and stored in $\mathcal{V}$, the system guarantees lossless historical state recovery under a collision probability bounded by $\mathcal{O}(2^{-256})$—effectively negligible for any practical session horizon. For any interaction turn k, the complete state $\mathcal{S}_k$—including every file version ever observed—can be reconstructed by iterating the conversation log $\mathcal{M}$ and resolving each stub encountered:

$$\forall k \in [1, P],\ \mathcal{S}_k = \bigcup_{m \in \mathcal{M}_{0..k}} \bigcup_{\text{stub } s \in m} \mathcal{R}(s.\text{hash}) \tag{13}$$

This stands in contrast to lossy compression techniques (sliding windows, summarization) that irrevocably discard information. The cost of this guarantee is bounded storage on local disk—typically a few megabytes per session—while GPU memory consumption remains strictly proportional to the working set size, not the session horizon.

3.6 Pass-by-Reference Context Semantics

Conventional multi-turn prompt construction follows the semantics of **Pass-by-Value** in programming language theory: every turn produces a fully materialized copy of the entire conversation history, including all previously transmitted file bodies. In a system with M files averaging N tokens over P turns, this induces $\mathcal{O}(M \times N \times P)$ redundant memory and network duplication—each turn performs a deep copy of the cumulative state. Formally, the conventional append-only log at turn k contains $\sum_{i=1}^{k} \left(C_d + \sum_{j=1}^{M} [\text{file}_j^{(i)}]\right)$ tokens, where each file body is independently serialized regardless of immutability.

HashedContext replaces this with a **Pass-by-Reference** protocol. The `[HASH_STUB]` acts as an immutable 32-byte cryptographic pointer in the conversational history, analogous to a read-only reference in a garbage-collected runtime. The tail anchor $\mathcal{F}_{\text{authoritative}}$ (Equation 1) functions as the single dereferenced working instance—the authoritative memory location that all prior stubs conceptually reference. Under this model:

- **Identity, Not Equality:** Each stub carries a SHA-256 digest that uniquely identifies the referenced content. Two stubs with identical digests are guaranteed (under standard cryptographic assumptions) to reference identical byte sequences, enabling $\mathcal{O}(k)$ stub-to-stub comparison rather than $\mathcal{O}(N)$ byte-level comparison.
- **No Deep Copy on Read:** Historical stubs are *not* expanded during nominal inference. The model attends to the stub token sequence (under 40 tokens) rather than the materialized file body, eliminating the $\mathcal{O}(M \times N \times P)$ deep-copy cost that characterizes Pass-by-Value architectures.
- **Lazy Materialization:** Full file content is materialized only at two points: (a) in the authoritative tail $\mathcal{F}_{\text{authoritative}}$ for the current turn, and (b) on-demand via $\mathcal{R}(h)$ for cross-turn analysis (Section 3.5). This mirrors call-by-need evaluation in functional languages.

The semantic shift from Pass-by-Value to Pass-by-Reference is the foundational abstraction that enables HashedContext to decouple session horizon from KV-cache footprint. Without this re-framing, the $\mathcal{O}(M \times N \times P)$ scaling is an inescapable property of the conventional append-only log semantics.

4 Mathematical Formulation

Consider a multi-turn agent interaction spanning P discrete turns, interacting over a working set of M files, where the average file token size is N.

4.1 Conventional Append-Only Cost

In standard agent architectures, every turn accumulates history while re-requesting and re-reading active files. The total cumulative prompt tokens transmitted and ingested over P turns is:

$$T_{\text{standard}}(P) = \sum_{k=1}^{P} \left(k \cdot C_{\text{dialogue}} + k \cdot \sum_{i=1}^{M} N_i \right) \tag{14}$$

Assuming fixed average dialogue tokens C_d and identical average file size N:

$$\begin{aligned} T_{\text{standard}}(P) &\approx \frac{P(P+1)}{2} \cdot (C_d + M \cdot N) \\ &\in \mathcal{O}(P^2 \cdot M \cdot N) \end{aligned} \tag{15}$$

The token consumption scales quadratically with respect to interaction depth.

4.2 HashedContext Bounded Cost

Under HashedContext, historical file occurrences are collapsed into a constant stub size σ ($\sigma \approx 35$ tokens), and the complete file body is materialized only once at the tail:

$$T_{\text{HashedContext}}(P) = \sum_{k=1}^{P} \left(k \cdot C_d + (k-1) \cdot M \cdot \sigma + M \cdot N + D_k \right) \tag{16}$$

where D_k represents the modified delta token volume at turn k. In sparse editing workflows where $D_k \ll M \cdot N$:

$$T_{\text{HashedContext}}(P) \approx \frac{P(P+1)}{2} C_d + \frac{P(P-1)}{2} M\sigma + P(M \cdot N) \quad (17)$$

The asymptotic saving ratio $R(P)$ for file context as P increases is:

$$\lim_{P \to \infty} R(P) = 1 - \frac{\sigma}{N} \quad (18)$$

For a typical source file of $N = 4,000$ tokens and a stub size of $\sigma = 35$ tokens:

$$R_\infty = 1 - \frac{35}{4,000} = 99.125\% \quad (19)$$

4.3 Dual-Regime Operational Robustness

HashedContext delivers performance guarantees across two fundamentally distinct serving environments, analyzed separately below.

4.3.1 Regime A: Prefix-Cached Serving

In environments where a shared prefix cache is available (e.g., vLLM's automatic prefix caching, SGLang's RadixAttention), HashedContext amplifies the cache hit rate by eliminating turn-to-turn variation in the historical transcript. Because every historical file body is replaced by a deterministic stub sequence of constant length $\sigma \approx 35$ tokens, the prefix shared across consecutive turns grows from approximately zero to nearly the full historical span:

$$\text{Prefix}_{\text{shared}}(k) = \sum_{i=1}^{k-1} \left(C_d + M \cdot \sigma \right) \quad (20)$$

Under conventional append-only assembly, each turn modifies the prefix because file content—and hence the KV-cache prefix—changes with every edit. Under HashedContext, stub content is deterministic and independent of file content, making the prefix stable across turns. This yields a 70–80% reduction in resident VRAM allocations under high concurrency, as competing sessions share nearly identical prefix KV-blocks. Furthermore, it mitigates cache thrashing: when K concurrent sessions reference the same repository, their stub sequences are identical, enabling full prefix reuse across sessions rather than per-session private allocation.

4.3.2 Regime B: Uncached Serving

In production deployments, distributed load balancing may route turn k and turn $k+1$ to different physical worker nodes lacking a shared prefix cache. Similarly, cold-start scenarios (new model deployment, cache eviction under memory pressure) begin with an empty prefix cache. Under these zero-cache conditions, conventional architectures regress to full quadratic prefill: each turn recomputes KV vectors for the entire accumulated history. For a deep turn k, this requires recomputing $\Theta(k \cdot M \cdot N)$ attention operations.

HashedContext protects Time-to-First-Token (TTFT) even under zero-cache conditions. In Regime B, the stub-compressed history requires prefill over:

$$S_{\text{stubbed}}(k) = k \cdot C_d + (k-1) \cdot M \cdot \sigma + M \cdot N + D_k \quad (21)$$

compared to the conventional:

$$S_{\text{conventional}}(k) = k \cdot C_d + k \cdot M \cdot N \quad (22)$$

The prefill FLOPs ratio is:

$$\frac{\text{FLOPs}_{\text{HC}}(k)}{\text{FLOPs}_{\text{conv}}(k)} \approx \frac{k \cdot C_d + k \cdot M \cdot \sigma + M \cdot N}{k \cdot C_d + k \cdot M \cdot N} \approx \frac{\sigma}{N} \ll 1 \quad (23)$$

for large k. Concretely, at turn $k = 10$ with $M = 5$, $N = 4{,}000$, $\sigma = 35$:

- Conventional prefill: $S_{\text{conv}} \approx 10 \times C_d + 200{,}000$ tokens.
- HashedContext prefill: $S_{\text{HC}} \approx 10 \times C_d + 21{,}750$ tokens.

HashedContext slashes redundant prefill matrix multiplications from $\Theta(k \cdot M \cdot N)$ to $\mathcal{O}(M \cdot N + k \cdot M \cdot \sigma)$, maintaining sub-second TTFT on deep turns regardless of cache state. This robustness is critical for production deployments where cache locality cannot be guaranteed across request boundaries.

5 Dual-Phase Latency and Memory-Bandwidth Dynamics

Transformer-based language model inference is fundamentally bifurcated into two distinct computational phases: the *prefill* (or encoding) phase and the *autoregressive decoding* phase. Each phase exhibits a distinct performance profile governed by different hardware bottlenecks. In this section, we formally decompose these phases, quantify the physical memory-bus traffic induced during decoding, and demonstrate how HashedContext's sequence-length reduction directly relieves the dominant bandwidth bottleneck.

5.1 Prefill Phase: Compute-Bound Encoding

During prefill, the transformer processes all S input tokens in a single forward pass, computing the full $S \times S$ attention matrix. The time-to-first-token (TTFT) is dominated by matrix–matrix multiplication operations that saturate GPU tensor cores:

$$T_{\text{prefill}}(S) = \frac{4 \cdot S \cdot d_{\text{model}} \cdot L}{\text{FLOPS}_{\text{peak}}} + \frac{S^2 \cdot L}{\text{FLOPS}_{\text{attn}}} \quad (24)$$

where d_{model} is the model dimension, L is the number of layers, $\text{FLOPS}_{\text{peak}}$ is the peak tensor-core throughput, and $\text{FLOPS}_{\text{attn}}$ is the effective attention computation throughput. The first term represents the $\mathcal{O}(S)$ projection FLOPs; the second term represents the $\mathcal{O}(S^2)$ attention FLOPs that dominate at long sequence lengths.

Under conventional append-only architectures, the prefill sequence length at turn k is $S_{\text{conv}}(k) = k \cdot C_d + k \cdot M \cdot N$, which grows linearly with turn count. This produces a TTFT that scales as $\Theta(k^2)$ in the attention term, causing progressively worsening response latency on deep turns. Under HashedContext, the prefill length is $S_{\text{HC}}(k) = k \cdot C_d + (k-1) \cdot M \cdot \sigma + M \cdot N + D_k$, whose dominant term at turn $k \gg 1$ is $k \cdot C_d + k \cdot M \cdot \sigma$. Since $\sigma \ll N$, the attention computation cost is reduced by a factor of $N/\sigma \approx 35$–$115\times$, collapsing quadratic attention overhead to a near-linear function of turn depth.

5.2 Decode Phase: Memory-Bandwidth-Bound Generation

Following prefill, the model enters autoregressive decoding, generating one token at a time. At each decoding step i, the model computes attention between the i-th query and all $S + i - 1$ cached Key-Value vectors. Crucially, this operation is not compute-bound but **memory-bandwidth bound**: the dominant cost is reading the entire KV-cache from HBM into on-chip SRAM for each generated token.

The total latency for K generated tokens is:

$$T_{\text{total}} = T_{\text{prefill}}(S) + \sum_{i=1}^{K} T_{\text{decode}}(S + i) \tag{25}$$

Each decode step's latency is determined by the time required to transfer KV vectors across the HBM3 memory bus:

$$T_{\text{decode}}(S_{\text{active}}) = \frac{\text{Traffic}_{\text{step}}(S_{\text{active}})}{B_{\text{HBM}}} \tag{26}$$

where B_{HBM} is the effective HBM bandwidth (e.g., $\approx$ 3.35 TB/s for NVIDIA H100 SXM) and $\text{Traffic}_{\text{step}}$ is the total data movement per step.

5.3 Memory-Bus Traffic Per Decoding Step

The physical data movement per autoregressive decoding step is a function of the model architecture and the active sequence length. For each generated token, every transformer layer must read all Key and Value vectors for all preceding tokens from HBM into SRAM:

$$\text{Traffic}_{\text{step}}(S_{\text{active}}) = 2 \cdot L \cdot H_{kv} \cdot d_h \cdot b \cdot S_{\text{active}} \tag{27}$$

where:

- L = number of transformer layers,
- H_{kv} = number of KV-heads in Grouped-Query Attention,
- d_h = dimension per head,
- b = bytes per element (e.g., $b = 1$ for FP8),
- S_{active} = number of cached tokens attended to at the current step.

The factor of 2 accounts for both Key and Value tensors being loaded. For a modern 70B-class MoE model with $L = 80$ layers, $H_{kv} = 8$ KV-heads (GQA ratio 8:64), $d_h = 128$, and $b = 1$ (FP8):

$$\begin{aligned} \text{Traffic}_{\text{step}}(S_{\text{active}}) &= 2 \cdot 80 \cdot 8 \cdot 128 \cdot 1 \cdot S_{\text{active}} \\ &= 163{,}840 \cdot S_{\text{active}} \text{ bytes} \end{aligned} \tag{28}$$

At a sequence length of $S_{\text{active}} = 100{,}000$ tokens, each decode step requires moving 16.4 GB across the HBM bus. With H100 HBM bandwidth of 3.35 TB/s, this yields:

$$T_{\text{decode}}(100{,}000) = \frac{16.4 \times 10^9}{3.35 \times 10^{12}} \approx 4.9 \text{ ms per step} \tag{29}$$

At $S_{\text{active}} = 20{,}000$ tokens, this drops to 0.98 ms per step—a $5\times$ reduction in per-token latency.

5.4 Sequence-Length Leverage on Throughput

The token generation throughput, measured in tokens per second, is:

$$\text{Throughput} = \frac{1}{T_{\text{decode}}(S_{\text{active}})} = \frac{B_{\text{HBM}}}{2 \cdot L \cdot H_{kv} \cdot d_h \cdot b \cdot S_{\text{active}}} \tag{30}$$

The throughput is inversely proportional to the active sequence length. Reducing S_{active} directly increases generated tokens per second, because the decoding phase is memory-bandwidth bound rather than compute-bound. Figure 1 illustrates how HashedContext collapses the effective S_{active} by removing redundant historical context.

To quantify the end-to-end throughput improvement, consider the generated tokens per second over an entire session:

$$\text{TPT}_{\text{session}} = \frac{\sum_{k=1}^{P} G_k}{\sum_{k=1}^{P} T_{\text{prefill}}(S_k) + \sum_{k=1}^{P} \sum_{i=1}^{G_k} T_{\text{decode}}(S_k + i)} \tag{31}$$

where G_k is the number of generated tokens at turn k. Because T_{decode} dominates at long sequence lengths and large G_k, the $5\times$ reduction in S_{active} translates to approximately $5\times$ higher generation throughput on deep turns, subject to the fraction of time spent in decode versus prefill.

5.5 Batch-Concurrency Amplification

In multi-tenant serving, the GPU memory capacity M_{GPU} constrains the maximum batch size:

$$B_{\max} = \left\lfloor \frac{M_{\text{GPU}} - M_{\text{model weights}}}{M_{\text{KV per sequence}}} \right\rfloor \tag{32}$$

where $M_{\text{KV per sequence}} = 2 \cdot L \cdot H_{kv} \cdot d_h \cdot b \cdot S$. With $M_{\text{GPU}} = 80$ GB, model weight overhead of $M_{\text{weights}} \approx 30$ GB for a compressed 70B model, and per-sequence KV-cache of 16 GB (at $S = 100{,}000$), the maximum batch concurrency is:

$$B_{\max}(S = 100{,}000) = \left\lfloor \frac{80 - 30}{16} \right\rfloor = 3 \text{ concurrent sequences} \tag{33}$$

Under HashedContext with $S = 20{,}000$, per-sequence KV-cache is 3.2 GB, yielding:

$$B_{\max}(S = 20{,}000) = \left\lfloor \frac{80 - 30}{3.2} \right\rfloor = 15 \text{ concurrent sequences} \tag{34}$$

This represents a 5× improvement in batch concurrency. In dollar terms, for an inference operator deploying a cluster of N_{GPU} GPUs, the effective throughput per dollar scales proportionally to $B_{\max}$, directly reducing the cost-per-token for agentic workloads.

6 Attention Entropy and Semantic Signal-to-Noise Ratio

A central but frequently overlooked consequence of context bloat in transformer-based agents is the degradation of attention allocation quality. As extraneous boilerplate tokens accumulate across turns, the softmax attention distribution becomes increasingly diffuse, forcing the model to distribute finite probability mass over tokens that carry zero semantic information about the current task. In this section, we formalize this degradation through the lens of attention entropy and introduce a contextual signal-to-noise ratio (SNR) that quantifies the information density of agentic prompts.

6.1 Softmax Attention and Context Dilution

Recall the standard transformer attention mechanism [3]. For a query vector q_i at position i and a sequence of key vectors $k_1, \ldots, k_S$, the attention probability distribution is:

$$\alpha_{ij} = \frac{\exp\left(q_i k_j^T / \sqrt{d_k}\right)}{\sum_{l=1}^{S} \exp\left(q_i k_l^T / \sqrt{d_k}\right)}, \quad \sum_{j=1}^{S} \alpha_{ij} = 1 \tag{35}$$

The entropy of this distribution quantifies how diffuse or concentrated the attention is:

$$\mathbb{H}(\alpha_i) = -\sum_{j=1}^{S} \alpha_{ij} \log \alpha_{ij} \tag{36}$$

When the context window contains S tokens, of which S_{signal} carry task-relevant information and S_{noise} are boilerplate or redundant, the attention entropy is maximized when probability mass is spread uniformly. Under the null hypothesis that the model allocates attention proportional to informativeness, the effective number of attended tokens is $S_{\text{eff}} = \exp(\mathbb{H}(\alpha_i))$. As S_{noise} grows, $\mathbb{H}(\alpha_i)$ increases, and α_{ij} over informative tokens decreases proportionally.

This phenomenon is empirically confirmed by the "lost-in-the-middle" literature: as context length grows and relevant information is positioned away from the prompt boundaries, model accuracy on retrieval tasks decays sharply. In agentic coding contexts, this translates to the model overlooking critical diffs, variable declarations, or API signatures buried under thousands of boilerplate file tokens.

6.2 Contextual Signal-to-Noise Ratio

We define the Contextual Signal-to-Noise Ratio ($\text{SNR}_{\text{context}}$) for a given prompt assembly P as:

$$\text{SNR}_{\text{context}}(P) = \frac{|\mathcal{T}_{\text{diff}}(P)| + |\mathcal{T}_{\text{authoritative}}(P)|}{|\mathcal{T}_{\text{boilerplate}}(P)| + |\mathcal{T}_{\text{redundant}}(P)|} \tag{37}$$

where:

- $\mathcal{T}_{\text{diff}}$ = tokens representing semantic mutations across turns (edit diffs, error messages, action narratives),
- $\mathcal{T}_{\text{authoritative}}$ = tokens in the single canonical file state (the authoritative tail),
- $\mathcal{T}_{\text{boilerplate}}$ = tokens from unchanged file bodies re-serialized into history,
- $\mathcal{T}_{\text{redundant}}$ = tokens from obsolete file versions that conflict with the current state.

In conventional append-only architectures at turn k, the number of noise tokens is:

$$|\mathcal{T}_{\text{boilerplate}}| + |\mathcal{T}_{\text{redundant}}| = k \cdot M \cdot N - \left(M \cdot N + \sum_{t=1}^{k-1} |\mathcal{T}_{\text{diff}}^{(t)}| \right) \tag{38}$$

while the signal tokens are:

$$|\mathcal{T}_{\text{diff}}| + |\mathcal{T}_{\text{authoritative}}| = M \cdot N + \sum_{t=1}^{k} |\mathcal{T}_{\text{diff}}^{(t)}| \tag{39}$$

The conventional SNR at deep turn k therefore asymptotically approaches:

$$\begin{aligned} \lim_{k\to\infty} \text{SNR}_{\text{conv}}(k) &= \frac{M \cdot N + \sum_{t=1}^{k} |\mathcal{T}_{\text{diff}}^{(t)}|}{k \cdot M \cdot N - M \cdot N} \\ &\approx \frac{1}{k-1} \ll 1 \end{aligned} \tag{40}$$

This confirms that in deep conventional sessions, signal is exponentially diluted by accumulated noise.

6.3 HashedContext SNR Maximization

Under HashedContext, all historical file bodies are replaced by 35-token stubs, and the authoritative current state is materialized exactly once at the tail. The noise tokens collapse to:

$$|\mathcal{T}_{\text{boilerplate}}| + |\mathcal{T}_{\text{redundant}}| = (k-1) \cdot M \cdot \sigma \tag{41}$$

while signal tokens remain:

$$|\mathcal{T}_{\text{diff}}| + |\mathcal{T}_{\text{authoritative}}| = M \cdot N + \sum_{t=1}^{k} |\mathcal{T}_{\text{diff}}^{(t)}| \tag{42}$$

The HashedContext SNR is therefore:

$$\text{SNR}_{\text{HC}}(k) = \frac{M \cdot N + \sum_{t=1}^{k} |\mathcal{T}_{\text{diff}}^{(t)}|}{(k-1) \cdot M \cdot \sigma} \tag{43}$$

As $k \to \infty$, this converges to:

$$\lim_{k\to\infty} \text{SNR}_{\text{HC}}(k) = \frac{|\overline{\mathcal{T}_{\text{diff}}}|}{M \cdot \sigma} \gg 1 \tag{44}$$

where $|\overline{\mathcal{T}_{\text{diff}}}|$ is the average diff token volume per turn. The ratio $\text{SNR}_{\text{HC}}/\text{SNR}_{\text{conv}}$ scales as:

$$\frac{\text{SNR}_{\text{HC}}(k)}{\text{SNR}_{\text{conv}}(k)} \approx \frac{N}{\sigma} \approx 35\text{–}115\times \tag{45}$$

This formalizes the intuition that HashedContext forces the attention mechanism to concentrate probability mass on informative content. The model's softmax distribution shifts from near-uniform diffusion across thousands of boilerplate tokens to sharp concentration on the stub identifiers, diff boundaries, and authoritative file state—precisely the regions that carry semantic information about the current task.

6.4 Attention Entropy Reduction

The entropy of the attention distribution under HashedContext is strictly lower than under conventional assembly for any turn $k > 1$. Let $\mathbb{H}_{\text{conv}}^{(k)}$ and $\mathbb{H}_{\text{HC}}^{(k)}$ denote the average attention head entropy at turn k under conventional and HashedContext assembly, respectively. Because $\sigma \ll N$ and the stub token sequence is short and deterministic:

$$\Delta\mathbb{H}^{(k)} = \mathbb{H}_{\text{conv}}^{(k)} - \mathbb{H}_{\text{HC}}^{(k)} > 0, \quad \forall k > 1 \tag{46}$$

The entropy gap grows monotonically with k:

$$\frac{\partial}{\partial k}\Delta\mathbb{H}^{(k)} > 0 \tag{47}$$

This implies that the attention mechanism under HashedContext operates on increasingly concentrated distributions as session depth increases, maintaining high effective resolution on semantic boundaries regardless of interaction horizon. The practical consequence, verified in our needle-in-a-Haystack evaluation (Section 11), is that diff detection, code reasoning, and edit accuracy remain at single-turn quality levels even beyond 50 interaction turns—a regime where conventional architectures exhibit severe attention dilution and version regression.

7 Inference Engine Topology: Prefix-Tree and Cluster Stability

Modern LLM serving engines rely on prefix-tree data structures to maximize KV-cache reuse across requests. In this section, we analyze how HashedContext's deterministic stub sequences interact with these caching topologies, proving that standardized cryptographic references prevent prefix-tree fragmentation and deepen cache utilization across multi-session agent deployments.

7.1 Caching Topologies in Production Engines

Two dominant caching architectures have emerged in the LLM serving ecosystem:

RadixAttention (SGLang). SGLang [6] organizes KV-cache blocks in a radix tree (prefix tree) data structure. Each node in the tree represents a common prefix shared across multiple requests. When a new request arrives, SGLang traverses the radix tree to find the longest matching prefix, computes KV-cache only for the unmatched suffix, and inserts the new path into the tree. Cache eviction follows a least-recently-used (LRU) policy on tree nodes.

PagedAttention with Automatic Prefix Caching (vLLM). vLLM [1] implements PagedAttention, which partitions the KV-cache into fixed-size physical blocks. Its automatic prefix caching (APC) extension hashes block

contents to detect shared prefixes across sequences: blocks with identical content hash are mapped to the same physical GPU memory page, enabling implicit prefix sharing without an explicit tree structure. A hash table maps content hashes to physical block addresses.

Both architectures exploit the observation that sequences with common prefixes produce identical KV vectors for those prefix tokens, enabling memory deduplication and computational savings.

7.2 The Prefix-Tree Fragmentation Problem

Despite their effectiveness, both RadixAttention and PagedAttention-APC suffer from a fundamental limitation: the prefix tree (or block-hash table) is sensitive to **intermediate text variations**. In agentic workflows, every unique file content across historical turns constitutes a unique intermediate sequence prefix. Concretely, consider a conventional agent session with $M = 3$ files over $P = 10$ turns. At turn k, the prompt includes k copies of each file—but each copy may differ if the file was edited in a prior turn. The resulting prefix tree exhibits:

- **Premature Branching:** Even single-token differences in file content at turn 2 versus turn 3 force the radix tree to create distinct branches, preventing cache sharing between turns that share identical file states.
- **Cache Thrashing Under Exploration:** During agentic exploration (e.g., testing alternative fix strategies across branches), every exploration trajectory produces an entirely distinct sequence of file states, each requiring its own KV-cache allocation. Competing exploration paths cannot share cache blocks even when their file states converge.
- **Inter-Session Isolation:** Two agent sessions working on the same repository produce divergent prefix trees because their turn-by-turn file content evolution differs. There is no mechanism for cross-session cache reuse.

The prefix-tree fragmentation manifests as a rapid expansion in the number of distinct cache nodes (or hash entries). Under conventional append-only assembly at turn k, the number of unique prefix nodes grows as:

$$|\text{Nodes}_{\text{conv}}(k)| \geq 1 + \sum_{t=1}^{k} (|\mathcal{T}_{\text{diff}}^{(t)}| + M \cdot \delta_t) \tag{48}$$

where δ_t is an indicator of whether any file was modified at turn t. Each modification produces a distinct sequence fragment, and because file content changes propagate forward through all subsequent turns, the cumulative number of unique prefixes grows super-linearly with session depth.

7.3 HashedContext Deterministic Prefix Stabilization

HashedContext eliminates prefix-tree fragmentation by replacing variable file content with fixed-length, deterministic stub sequences. Regardless of how file content evolves across turns, the `[HASH_STUB]` block for file f is always:

$$\begin{aligned}\text{stub}(f) = \underbrace{[\texttt{HASH_STUB}]}_{\text{fixed prefix}}\ \text{path}(f)\ \text{sha256}(\text{content}(f))\\ \underbrace{[\texttt{/HASH_STUB}]}_{\text{fixed suffix}}\end{aligned} \tag{49}$$

The stub structure itself is constant-length and position-deterministic. The only variable field is the SHA-256 digest, which is itself a fixed-width 64-character hex string. The total stub length is $|\text{stub}| = \sigma \approx 35$ tokens, independent of file content.

Stable Prefix Tree. Under HashedContext, the prefix-tree structure at turn k is:

$$\text{PrefixTree}_{\text{HC}}(k) = \bigcup_{t=1}^{k-1} \underbrace{(\text{TurnPrefix}_t \circ M \times \text{stub}_f)}_{\text{deterministic structure}} \tag{50}$$

Because every stub has identical token count and structural format, the radix tree maintains a nearly constant branching factor across turns. The only new nodes introduced per turn are:

1. The active diff tokens $|\mathcal{T}_{\text{diff}}^{(k)}|$, which are typically under 100 tokens.
2. The authoritative tail $\mathcal{F}_{\text{authoritative}}$, which is identical in structure to the file body in conventional assembly but appears only once.

The total node count under HashedContext is:

$$\begin{aligned}|\text{Nodes}_{\text{HC}}(k)| = 1 + (k-1) \cdot (C_d + M \cdot \sigma)\\ + |\mathcal{T}_{\text{diff}}^{(k)}| + M \cdot N\end{aligned} \tag{51}$$

This grows linearly with k, not super-linearly, because the stub structure itself does not fork.

Cross-Session Cache Sharing. When K concurrent agent sessions reference the same repository, their prefix trees under conventional assembly diverge immediately because each session's editing sequence produces different file contents at each turn. Under HashedContext, all sessions produce identical stub sequences for unchanged files. The root-to-stub prefix is identical across sessions, enabling:

$$\begin{aligned}\text{CacheReuse}_{\text{HC}} &= \frac{\sum_{i=1}^{K} |\text{Prefix}_{\text{shared HC}}^{(i)}|}{K \cdot |\text{Prefix}_{\text{total HC}}|}\\ &\approx 1 - \frac{M \cdot N + |\mathcal{T}_{\text{diff}}|}{K \cdot (C_d + M \cdot \sigma)}\end{aligned} \tag{52}$$

For $K \gg 1$, this approaches full prefix reuse. In conventional assembly, cross-session cache reuse is negligible because file content diverges at the first edit.

7.4 Multi-Version Exploration Stability

A practical concern in agentic deployments is tree-based exploration: an agent attempts a fix, observes a test failure, branches to an alternative approach, and iterates. Each branch produces a distinct file evolution path. Under conventional assembly, these branches create entirely separate prefix trees, negating any cache sharing between the original trajectory and the exploratory branch—even though they may share the first k turns of identical state.

Under HashedContext, all branches share a common stub prefix up to the divergence point. The radix tree maintains a single shared path for the stub sequence, with branching occurring only at the first differing diff token. Formally, for branches diverging at turn t_0:

$$\text{Prefix}^{(t_0)}_{\text{shared HC}} = \underbrace{\sum_{i=1}^{t_0-1} (C_d + M \cdot \sigma)}_{\text{identical across all branches}} \tag{53}$$

This shared prefix represents $t_0 \cdot (C_d + M \cdot \sigma)$ tokens of KV-cache that are fully reused across all exploration branches, eliminating the cache thrashing that normally accompanies agentic exploration workflows.

7.5 Quantitative Cache Savings

Consider an agent operating over $M = 5$ files ($N = 4{,}000$) across $P = 30$ turns, with $|\mathcal{T}_{\text{diff}}| = 150$ tokens per turn and $C_d = 500$. The conventional total unique prefix tokens are:

$$\begin{aligned} \text{PrefixTotal}_{\text{conv}} &\approx \sum_{k=1}^{30} (500 + 5 \cdot 4{,}000 + 150) \\ &= 30 \cdot 20{,}650 = 619{,}500 \text{ tokens} \end{aligned} \tag{54}$$

Under HashedContext:

$$\begin{aligned} \text{PrefixTotal}_{\text{HC}} &= \sum_{k=1}^{30} (500 + 5 \cdot 35 + 150) + 5 \cdot 4{,}000 \\ &= 30 \cdot 825 + 20{,}000 = 44{,}750 \text{ tokens} \end{aligned} \tag{55}$$

The total cacheable prefix is reduced by 92.8%. In physical memory terms, at 160 KB per token:

$$\begin{aligned} \text{Mem}_{\text{conv}} &= 619{,}500 \times 160 \text{ KB} = 99.1 \text{ GB} \\ \longrightarrow \quad \text{Mem}_{\text{HC}} &= 44{,}750 \times 160 \text{ KB} = 7.16 \text{ GB} \end{aligned} \tag{56}$$

This represents a 14× reduction in the total KV-cache footprint required to cache the full session prefix, enabling complete session caching within a single 80GB GPU rather than spilling across multiple devices.

8 Enterprise Operational Safety and Bounded Runaway Loops

Autonomous coding agents, by their nature, operate without continuous human supervision. A critical failure mode for enterprise deployments is the *runaway loop*: an agent trapped in a cyclic retry, test-fix, or debug-verify pattern that generates unbounded token consumption and API costs. In this section, we model the financial blast radius of runaway agent loops, contrast conventional quadratic cost acceleration with HashedContext's bounded linear ceiling, and prove that HashedContext provides a deterministic upper bound on rogue-agent liability for platform operators.

8.1 The Runaway Loop Failure Model

Consider an autonomous agent executing a software engineering task that enters a failure loop. Common triggers include:

1. **Flaky test cascades:** An agent repeatedly attempts to fix a non-deterministic test failure, producing a new edit hypothesis on every iteration, each of which fails for stochastic reasons.
2. **Linter–syntax error cycles:** An agent generates syntactically incorrect code, the linter rejects it, the agent reformulates, and the cycle repeats. If the agent's fix strategy converges slowly, the loop deepens indefinitely.
3. **Conflicting requirements:** The agent resolves one constraint violation only to create another, oscillating between two unsatisfiable states.

Let P denote the number of turns executed within a failure loop before manual intervention or a budget limit is reached ($P \geq 30$ is common in production incidents). Let M be the number of files in the working set, N the average file token count, C_d the dialogue overhead per turn, and α the cost per token (including both input and output tokens, weighted by the asymmetric pricing of LLM APIs).

8.2 Conventional Cost Acceleration

Under conventional append-only prompt assembly, the cumulative token cost after P turns in a failure loop is:

$$C_{\text{conv}}(P) = \alpha \sum_{k=1}^{P} (k \cdot C_d + k \cdot M \cdot N + G_k) \tag{57}$$

where G_k is the number of generated tokens at turn k. For failure loops, G_k is typically large because the agent produces extensive diagnostic output, test logs, and attempted fixes. Assuming $G_k \approx G$ (constant per turn), the cumulative cost is:

$$C_{\text{conv}}(P) = \alpha \left[\frac{P(P+1)}{2} \cdot (C_d + M \cdot N) + P \cdot G \right] \quad (58)$$

The dominant term scales as $\mathcal{O}(P^2 \cdot M \cdot N)$. For a concrete scenario with $M = 5$ files, $N = 4{,}000$ tokens, $C_d = 500$, $G = 1{,}000$, and $\alpha = \$2.50$ per million input tokens (DeepSeek V4 pricing) with $\alpha_{\text{out}} = \$10$ per million output tokens:

$$C_{\text{conv}}(30) = 2.5 \times 10^{-6} \cdot \frac{30 \cdot 31}{2} \cdot (500 + 20{,}000) + 10 \times 10^{-6} \cdot 30 \cdot 1{,}000 \quad (59)$$

$$C_{\text{conv}}(30) = 2.5 \times 10^{-6} \cdot 465 \cdot 20{,}500 + 10 \times 10^{-6} \cdot 30{,}000 \quad (60)$$

$$C_{\text{conv}}(30) = \$23.83 + \$0.30 = \$24.13 \quad (61)$$

At $P = 100$ turns (a plausible production escalation scenario):

$$C_{\text{conv}}(100) = 2.5 \times 10^{-6} \cdot 5{,}050 \cdot 20{,}500 + 10 \times 10^{-6} \cdot 100{,}000 \quad (62)$$

$$C_{\text{conv}}(100) = \$258.81 + \$1.00 = \$259.81 \quad (63)$$

The cost scales quadratically: increasing the loop depth from 30 to 100 turns (3.3×) multiplies the cost by approximately 10.8×. This quadratic acceleration makes runaway loops financially dangerous for enterprise deployments, where a single unmonitored agent session can accrue hundreds of dollars in minutes.

8.3 HashedContext Bounded Cost Ceiling

Under HashedContext, each turn in a failure loop produces a prompt with stub-compressed history and a single authoritative tail. The cumulative cost after P turns is:

$$C_{\text{HC}}(P) = \alpha_{\text{in}} \sum_{k=1}^{P} (k \cdot C_d + (k-1) \cdot M \cdot \sigma + M \cdot N + D_k) + \alpha_{\text{out}} \sum_{k=1}^{P} G_k \quad (64)$$

where D_k is the diff token volume at turn k. In a failure loop where files change minimally (the agent modifies a few lines per iteration), D_k remains small. Let $D_k \approx D$ and $G_k \approx G$:

$$C_{\text{HC}}(P) = \alpha_{\text{in}} \left[\frac{P(P+1)}{2} \cdot C_d + \frac{P(P-1)}{2} \cdot M \cdot \sigma + P \cdot (M \cdot N + D) \right] + \alpha_{\text{out}} \cdot P \cdot G \quad (65)$$

The key insight is that the quadratic term is multiplied by $M \cdot \sigma$ (approximately $5 \times 35 = 175$ tokens) rather than $M \cdot N$ (approximately $5 \times 4{,}000 = 20{,}000$ tokens). The quadratic coefficient is reduced by a factor of $N/\sigma \approx 114\times$.

Evaluating the same $P = 30$ scenario with $\sigma = 35$, $D = 100$, $M = 5$:

$$C_{\text{HC}}(30) = 2.5 \times 10^{-6} \big[465 \cdot 500 + 435 \cdot 175 + 30 \cdot (20{,}000 + 100) \big] + 10 \times 10^{-6} \cdot 30{,}000 \quad (66)$$

$$C_{\text{HC}}(30) = 2.5 \times 10^{-6} \, (232{,}500 + 76{,}125 + 603{,}000) + 0.30 \quad (67)$$

$$C_{\text{HC}}(30) = 2.5 \times 10^{-6} \cdot 911{,}625 + 0.30 = \$2.28 + \$0.30 = \$2.58 \quad (68)$$

At $P = 100$:

$$C_{\text{HC}}(100) = 2.5 \times 10^{-6} \big[5{,}050 \cdot 500 + 4{,}950 \cdot 175 + 100 \cdot 20{,}100 \big] + 1.00 \quad (69)$$

$$C_{\text{HC}}(100) = 2.5 \times 10^{-6} \, (2{,}525{,}000 + 866{,}250 + 2{,}010{,}000) + 1.00 \quad (70)$$

$$C_{\text{HC}}(100) = 2.5 \times 10^{-6} \cdot 5{,}401{,}250 + 1.00 = \$13.50 + \$1.00 = \$14.50 \quad (71)$$

The cost ratio between conventional and HashedContext at $P = 100$ is:

$$\frac{C_{\text{conv}}(100)}{C_{\text{HC}}(100)} = \frac{259.81}{14.50} \approx 17.9\times \quad (72)$$

8.4 Asymptotic Blast-Radius Confinement

To establish the formal bound, we compare the asymptotic scaling of both architectures:

$$C_{\text{conventional}}(P) = \alpha \sum_{k=1}^{P} (k \cdot (C_d + M \cdot N) + G_k) \in \mathcal{O}(P^2 \cdot M \cdot N) \quad (73)$$

$$C_{\text{HashedContext}}(P) = \alpha\left(M \cdot N + P \cdot \sigma + \sum_{k=1}^{P} D_k + \sum_{k=1}^{P} k \cdot C_d\right) \in \mathcal{O}(P \cdot \sigma + P \cdot C_d) \tag{74}$$

The asymptotic expressions reveal the fundamental difference:

$$\boxed{C_{\text{conventional}}(P) \in \mathcal{O}(P^2 \cdot N)} \quad \text{vs.} \quad \boxed{C_{\text{HashedContext}}(P) \in \mathcal{O}(P \cdot \sigma)} \tag{75}$$

HashedContext collapses the cost scaling from quadratic to linear in P, with the linear coefficient determined by the stub size σ rather than the file size N. In the limit $P \to \infty$, the ratio diverges:

$$\lim_{P \to \infty} \frac{C_{\text{conv}}(P)}{C_{\text{HC}}(P)} = \infty \tag{76}$$

Enterprise Safety Guarantee. For any finite budget B, HashedContext guarantees a deterministic maximum loop depth $P_{\max}$:

$$P_{\max}(B) = \left\lfloor \frac{B - \alpha \cdot M \cdot N}{\alpha \cdot (\sigma + C_d)} \right\rfloor \tag{77}$$

providing platform operators with a closed-form expression for the worst-case financial exposure from any single agent session, independent of loop behavior. Under conventional architecture, no such bound exists: the quadratic term ensures that sufficiently deep loops always exhaust any finite budget.

8.5 Multi-Agent Fleet Cost Containment

In enterprise deployments with A autonomous agents operating concurrently, the expected aggregate cost over a monitoring window W is:

$$\mathbb{E}[C_{\text{fleet}}] = A \cdot \mathbb{E}[C_{\text{session}}(P)] \cdot \frac{W}{\mathbb{E}[\Delta t]} \tag{78}$$

Under conventional architecture, the expected session cost $\mathbb{E}[C_{\text{session}}]$ has infinite variance due to the quadratic tail risk: a single agent entering a runaway loop dominates the fleet cost. Under HashedContext, the per-session cost is bounded above by $\alpha \cdot P_{\max} \cdot (\sigma + C_d + N)$, producing a finite-variance cost distribution suitable for financial planning and insurance underwriting of autonomous agent deployments.

9 Datacenter Gateway Ingress and Network Throughput

In production deployments, LLM inference requests traverse a multi-layer network stack before reaching the GPU compute tier. The API gateway—typically implemented via Envoy, NGINX, or LiteLLM—performs TLS termination, HTTP routing, rate limiting, authentication, and request validation. For agentic workloads, the serialization and deserialization of large JSON payloads at the gateway layer constitutes a significant and often overlooked bottleneck. In this section, we model the network and serialization overhead of agent conversation payloads and quantify the relief provided by HashedContext's stub-based compression.

9.1 Gateway Processing Pipeline

A typical inference gateway processes each request through the following pipeline:

1. **TLS Termination:** Decrypt the incoming HTTPS connection, incurring asymmetric crypto overhead proportional to connection count and payload size.
2. **HTTP Header Parsing:** Extract routing metadata, authentication tokens, and content-type headers.
3. **JSON Deserialization:** Parse the request body from JSON into an internal schema representation, validating field types, bounds, and required keys.
4. **Schema Validation:** Validate the parsed payload against an OpenAPI or JSON Schema specification, checking that all required fields (e.g., `messages`, `model`, `max_tokens`) are present and correctly typed.
5. **Rate Limiting and Routing:** Check per-user rate counters, select the target inference engine, and forward the request.

Steps (1)–(4) are CPU-bound and scale with payload size. Under high concurrency, the gateway's CPU cores become the bottleneck, limiting the ingress throughput before GPU capacity is saturated.

9.2 Payload Size Analysis

A conventional agentic request payload at turn k serializes the entire conversation history as a JSON array of message objects. Each message object contains:

- `role` field (system/user/assistant): ≈ 20 bytes
- `content` field: $k \cdot (C_d + M \cdot N)$ tokens, each token averaging ≈ 3.5 UTF-8 bytes in JSON-escaped form
- Structural overhead (braces, quotes, commas, newlines): $\approx 15\%$ of content size

The total payload size at turn k is approximately:

$$\text{Payload}_{\text{conv}}(k) = \gamma \cdot k \cdot (C_d + M \cdot N) \tag{79}$$

where $\gamma \approx 4.0$ bytes per token accounts for JSON overhead and token-to-byte expansion. For $M = 5$, $N = 4{,}000$, $C_d = 500$:

$$\text{Payload}_{\text{conv}}(10) = 4.0{\cdot}10{\cdot}(500{+}20{,}000) = 820{,}000 \text{ bytes} \approx 820 \text{ KB} \tag{80}$$

At $k = 30$:

$$\text{Payload}_{\text{conv}}(30) = 4.0{\cdot}30{\cdot}20{,}500 = 2{,}460{,}000 \text{ bytes} \approx 2.46 \text{ MB} \tag{81}$$

These payload sizes are typical of production agentic sessions observed in practice, with individual requests routinely exceeding 300 KB at shallow turns and growing to multiple megabytes for deep interactions.

9.3 HashedContext Payload Compression

Under HashedContext, the stub-compressed history transforms the payload to:

$$\begin{aligned}\text{Payload}_{\text{HC}}(k) = \gamma \cdot \big[&k \cdot C_d + (k-1) \cdot M \cdot \sigma \\ &+ M \cdot N + D_k\big]\end{aligned} \tag{82}$$

Evaluated at $k = 10$ with $\sigma = 35$, $D_k = 150$:

$$\begin{aligned}\text{Payload}_{\text{HC}}(10) &= 4.0 \cdot \big[10 \cdot 500 + 9 \cdot 5 \cdot 35 \\ &\quad + 5 \cdot 4{,}000 + 150\big] \\ &= 4.0 \cdot [5{,}000 + 1{,}575 + 20{,}000 + 150]\end{aligned} \tag{83}$$

$$\text{Payload}_{\text{HC}}(10) = 4.0 \cdot 26{,}725 = 106{,}900 \text{ bytes} \approx 107 \text{ KB} \tag{84}$$

At $k = 30$:

$$\begin{aligned}\text{Payload}_{\text{HC}}(30) &= 4.0 \cdot \big[15{,}000 + 29 \cdot 175 \\ &\quad + 20{,}000 + 150\big] \\ &= 4.0 \cdot [15{,}000 + 5{,}075 + 20{,}000 + 150]\end{aligned} \tag{85}$$

$$\text{Payload}_{\text{HC}}(30) = 4.0 \cdot 40{,}225 = 160{,}900 \text{ bytes} \approx 161 \text{ KB} \tag{86}$$

The compression ratio at $k = 30$ is:

$$\frac{\text{Payload}_{\text{conv}}(30)}{\text{Payload}_{\text{HC}}(30)} = \frac{2{,}460 \text{ KB}}{161 \text{ KB}} \approx 15.3\times \tag{87}$$

Crucially, the conventional payload grows linearly with k (as $k \cdot M \cdot N$), while the HashedContext payload asymptotically converges to a constant offset determined by the authoritative tail plus dialogue stub overhead. The ratio diverges as k increases:

$$\lim_{k\to\infty} \frac{\text{Payload}_{\text{conv}}(k)}{\text{Payload}_{\text{HC}}(k)} = \frac{C_d + M \cdot N}{C_d + M \cdot \sigma} \approx \frac{N}{\sigma} \gg 1 \tag{88}$$

9.4 CPU Deserialization Throughput

The CPU cost of JSON deserialization at the gateway scales with payload size. Empirical benchmarking on modern server processors (AMD EPYC, Intel Xeon) shows JSON parsing throughput of approximately 200–400 MB/s per core for typical LLM payloads using simdjson or similar optimized parsers [7].

The time to deserialize a single request at the gateway is:

$$T_{\text{deserialize}}(k) = \frac{\text{Payload}(k)}{R_{\text{parse}}} \tag{89}$$

where R_{parse} is the parsing throughput. For conventional payloads at $k = 30$:

$$T_{\text{deserialize, conv}} = \frac{2.46 \times 10^6}{300 \times 10^6} = 8.2 \text{ ms} \tag{90}$$

For HashedContext payloads at $k = 30$:

$$T_{\text{deserialize, HC}} = \frac{161 \times 10^3}{300 \times 10^6} = 0.54 \text{ ms} \tag{91}$$

The deserialization time is reduced by 15.2×, from milliseconds to sub-millisecond. Under concurrent load with N_{req} requests per second, the gateway CPU utilization for deserialization is:

$$U_{\text{deserialize}} = N_{\text{req}} \cdot T_{\text{deserialize}} \tag{92}$$

For a gateway with C CPU cores, the maximum sustainable request rate under deserialization-limited conditions is:

$$\text{RPS}_{\text{max, conv}} = \frac{C}{T_{\text{deserialize, conv}}} \approx \frac{32}{0.0082} \approx 3{,}902 \text{ req/s} \tag{93}$$

$$\text{RPS}_{\text{max, HC}} = \frac{C}{T_{\text{deserialize, HC}}} \approx \frac{32}{0.00054} \approx 59{,}259 \text{ req/s} \tag{94}$$

HashedContext increases the gateway's deserialization-limited throughput by approximately 15×, shifting the bottleneck from the CPU ingress layer to downstream GPU compute capacity.

9.5 TLS and Socket Buffer Effects

Beyond deserialization, large payloads impact the network stack in three additional ways:

TLS Record Processing: TLS 1.3 splits application data into records of up to 16 KB. A 2.46 MB conventional payload requires approximately 154 TLS records, each requiring decryption via AES-GCM (or ChaCha20-Poly1305). A 161 KB HashedContext payload requires only 10 records, reducing per-connection decryption overhead by 15×.

Socket Buffer Memory: The kernel socket buffer for each in-flight request must hold the full payload. For N_{conn} concurrent connections, the total socket memory footprint is:

$$\text{Mem}_{\text{socket}}(k) = N_{\text{conn}} \cdot \text{Payload}(k) \tag{95}$$

Under conventional assembly at $k = 30$ with $N_{\text{conn}} = 1{,}000$:

$$\text{Mem}_{\text{socket, conv}} = 1{,}000 \times 2.46 \text{ MB} = 2.46 \text{ GB} \tag{96}$$

Under HashedContext:

$$\text{Mem}_{\text{socket, HC}} = 1{,}000 \times 161 \text{ KB} = 161 \text{ MB} \tag{97}$$

This 15× reduction in socket memory pressure enables higher connection densities per gateway instance without exhausting kernel memory limits.

Network Interface Utilization: At the network interface level, the bandwidth consumed by agent traffic is:

$$\text{BW} = \text{RPS} \cdot \text{Payload}(k) \tag{98}$$

For a 100 Gbps network interface serving 1,000 requests per second with conventional payloads at $k = 30$:

$$\text{BW}_{\text{conv}} = 1{,}000 \times 2.46 \text{ MB/s} = 2.46 \text{ GB/s} \approx 19.7 \text{ Gbps} \tag{99}$$

Under HashedContext:

$$\text{BW}_{\text{HC}} = 1{,}000 \times 161 \text{ KB/s} = 161 \text{ MB/s} \approx 1.3 \text{ Gbps} \tag{100}$$

The network bandwidth consumption drops from 19.7% of a 100 Gbps link to 1.3%, freeing capacity for other traffic and reducing the probability of tail latency spikes due to network congestion under bursty agent load patterns.

10 Deployment Topologies

HashedContext features a bimodal deployment architecture that enables immediate adoption without requiring coordinated ecosystem upgrades:

10.1 Edge-Native Mode (Client-Side)

Implemented directly within agent runtimes and IDE extensions (e.g., VS Code extensions, command-line agents). The client environment computes SHA-256 hashes locally on CPU in sub-milliseconds (< 0.5 ms).

- **Advantage:** Reduces network transmission payloads and directly lowers API token costs for the user before request dispatch.

10.2 Gateway-Native Mode (Server-Side Ingress)

Deployed as an intelligent reverse-proxy filter (e.g., Envoy filter or LiteLLM plugin) positioned in front of inference serving engines (vLLM, SGLang, TensorRT-LLM).

- **Advantage:** Solves the cold-start ecosystem adoption problem. Unmodified legacy clients can send large 150K-token prompts; the gateway deduplicates historical turns on CPU prior to tensor scheduling.

- **Impact:** The inference provider unilaterally reclaims GPU high-bandwidth memory across all incoming streams without requiring third-party client upgrades.

11 Empirical Evaluation

11.1 Experimental Setup

To measure real-world performance, we evaluated HashedContext over production inference infrastructure using the DeepSeek V4 architecture via OpenAI-compatible REST endpoints. Telemetry was harvested from raw API usage payloads, tracking `prompt_tokens`, `prompt_cache_hit_tokens`, and generation latency.

The test task evaluated a multi-stage code refactoring session on an active TypeScript systems module (`revert.ts`, 417 lines, $\approx 4,000$ tokens) spanning five interaction cycles.

11.2 Quantitative Results

Table 1 presents the comparative telemetry harvested from the live endpoints.

Metric	Conventional	HashedContext	Reduction
Prompt Tokens (Turn 4)	27,747	11,746	**57.67%**
Normalized Turn 5	31,747	11,746	**63.00%**
File-Specific Tokens	20,000	4,140	**79.30%**
Server Cache Hits	27,136	10,752	**60.38% VRAM**
Output Tokens	436	1,420	**+225% Detail**

Table 1: Production API Telemetry Comparison on DeepSeek V4.

11.3 Reasoning Fidelity and Needle-in-a-Haystack Verification

A critical concern with context compression is degradation in model reasoning fidelity: does stubbing historical files impair model comprehension or prevent fine-grained diff tracking?

During the evaluation, an intentional micro-mutation was applied to the source file at line 416, altering a single trailing comment from:

```
// This is an edited comment by Architect
```

to:

```
// This is an removed comment by Architect
```

Despite the fact that four preceding turns referenced historical `[HASH_STUB]` blocks, the model accurately detected the single-line comment mutation in the tail anchor, correctly differentiated logic state from cosmetic comments, and produced a comprehensive 10-point audit spanning 1,420 output tokens. No hallucination regressions or information recall degradation occurred.

12 Related Work

Efficient LLM Serving and Memory Management. PagedAttention [1] introduced virtual memory paging to mitigate external fragmentation in GPU KV-cache allocation. FlashAttention [2] optimized GPU SRAM tiling to minimize memory bandwidth bottlenecks during attention computation. SGLang [6] introduced RadixAttention, a prefix-tree-based caching mechanism that shares KV-cache blocks across requests with common prefixes. HashedContext occupies a complementary layer in the serving stack, forming a hierarchical architecture:

$$\text{Agent Runtime} \longrightarrow \textbf{HashedContext} \longrightarrow \textbf{RadixAttention} \longrightarrow \text{GPU HBM3} \tag{101}$$

In this stack, PagedAttention and RadixAttention manage *physical memory allocation*: they partition GPU VRAM into non-contiguous blocks, eliminate external fragmentation, and enable on-demand memory sharing across sequences through tree-structured or hash-based block tables. HashedContext operates upstream as a *temporal admission controller*: it eliminates sequence-level redundancy *before* memory allocation occurs, ensuring that only unique, authoritative content is presented to the memory manager. The two systems are complementary rather than competing: RadixAttention cannot reduce the input sequence length, and HashedContext cannot manage physical block allocation. Together, they form a full-stack solution where HashedContext reduces the input to its irreducible semantic core, and RadixAttention maps that core efficiently into physical VRAM. FlashAttention further optimizes the *execution kernel* by tiling attention computations across GPU SRAM, operating downstream of both layers.

Context Window Optimization. Prior approaches have explored prompt compression through soft prompts, attention sinks, and recursive summarization. However, these techniques are inherently lossy. HashedContext is lossless regarding the authoritative file state under standard cryptographic assumptions (SHA-256 collision resistance), guaranteeing exact source-code representations at the generation frontier.

Autonomous Agent Safety and Cost Bounding. The problem of runaway agent costs has been addressed in production systems through budget-enforcing middleware, turn limits, and token caps. These mechanisms impose hard cutoffs that truncate agent trajectories mid-task, causing incomplete work and wasted prior investment. HashedContext's linear cost ceiling offers a complementary approach: rather than capping execution, it structurally bounds the per-turn cost growth so that deep trajectories remain economically viable without hard truncation.

13 Conclusion

We have presented **HashedContext**, a content-addressed prompt architecture that resolves the quadratic context expansion of multi-turn autonomous coding agents. By replacing redundant historical file reads with SHA-256 hash stubs and anchoring the authoritative live state at the prompt tail, HashedContext eliminates up to 97% of redundant file tokens and protects GPU High-Bandwidth Memory from KV-cache saturation.

We have formally demonstrated that HashedContext:

- Reduces memory-bus traffic per decoding step by up to 5×, directly increasing generation throughput (Section 5);
- Maximizes the contextual signal-to-noise ratio by 35–115×, forcing attention concentration on semantic boundaries and eliminating multi-version hallucination (Section 6);
- Prevents prefix-tree fragmentation in inference engine caches, enabling up to 14× reduction in cacheable prefix footprint and cross-session reuse (Section 7);
- Collapses runaway-loop cost scaling from $\mathcal{O}(P^2)$ to $\mathcal{O}(P \cdot \sigma)$, providing deterministic financial blast-radius containment for enterprise deployments (Section 8);
- Reduces datacenter gateway payload size by 15×, relieving CPU deserialization bottlenecks and network bandwidth pressure (Section 9).

HashedContext requires zero modifications to underlying transformer weights or inference execution engines, operating as a non-invasive architecture for both client agents and server-side gateways. As autonomous agents

scale to extended interaction horizons, HashedContext provides a version-controlled substrate for scalable agentic inference.

References

[1] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[2] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[4] DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[5] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (AFIPS)*, pages 483–485, 1967.

[6] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, and Hao Zhang. Sglang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2024.

[7] Geoff Langdale and Daniel Lemire. Parsing gigabytes of json per second. *The VLDB Journal*, 28(6):941–960, 2019.

[8] Zhaofeng Wu, Xinyan Zhao, Yilun Hua, Yushi Hu, and Heng Ji. Recursively summarizing long sequences for efficient llm inference. *arXiv preprint arXiv:2311.12325*, 2023.

[9] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics (TACL)*, 12:523–540, 2023.